MiniCPM-V 4.6 on-device latency — findings and the field-canvas approach (2026-09-30)

Follows `HANDOVER_MINICPM.md`. Device: Samsung SM-M356B (Exynos 1380, CPU only, 4 threads pinned to cores 4-7). Model: `minicpm-v-4.6-q4_k_m` + `mmproj-minicpm-v-4.6-q8_0`. Doc type: `pan_full`.

Status in one line: the app takes ~40 s per PAN scan today; a change already in the source cuts ~6 s; a new input approach (stitch the field regions into one image) takes ~22 s in the test harness but is **not built in the app** and its accuracy is **not yet good enough to ship**.

"Measured" = timed on the phone this session. "Estimated" = my reasoning, not run.

---

1. Where the app stands today (measured, in the app)

Run	Card	Image sent	Time	Real accuracy
1	UDAYRAJ SINGH	560x337	42.6 s	5/5
2	SAROJINI M	448x560	40.2 s	3/5 (name `SARJONIM M`, parent `MURIGESAN`)

The app's "PAN check: HIGH - 5/5 fields read" only checks that fields are present and well-formed. It does **not** check that the letters are right (run 2 passed it with two wrong names).

The image the app received in run 2 was an 848x1058 photo containing **both sides** of the card. The app shrank it to 449x560, so the front (all the fields) became a strip about 270 px tall and half the image went to the back of the card.

2. Where the 41 s goes (measured, verbose run, 560x350 input)

Stage	Time
Startup: fit pass, LM context, 3 GB KV cache (default 262,144-token context), projector load	~6.8 s
Vision encode: 3 tiles, ~7.3 s each	21.9 s
Image chunks decoded into the LM (~1.3 s each)	3.9 s
Text prefill + 65 generated tokens (3.7 s)	6.8 s
Teardown	0.9 s

Tokens: image = 60 (overview) + 63 + 63 = 186; total prompt 317; generated 65; total 382. For a 433x560 portrait input: grid 1x2, 63 + 60 + 60 = 183 image tokens, encode 22.2 s.

3. Why it is always 3 tiles (read from source)

`tools/mtmd/mtmd-image.cpp`, `get_slice_instructions` (line ~621) and `get_best_resize` (line ~769):

- The 448 comes from the model file (`image_size = 448`, printed as `load_hparams: image_size: 448`).
- If either side is > 448 the image is sliced; if both are <= 448 it is one tile.
- The grid never has a single slice (`get_best_grid` skips a count of 1, line ~847), so slicing always gives **2 slices + 1 overview = 3 tiles minimum**.
- Every tile is rescaled to an area of about 448x448, keeping aspect ratio and rounding each side to a multiple of 56 (14 px patch x 4 merge). `allow_upscale` is true for the slices, so a small input is *enlarged*: 433x560 became `refined size 560x672`. Tile sizes such as 392x504 are computed, not preset.
- One tile = 60-64 tokens = about 7-8 s in the vision encoder. Latency steps with tile count, not smoothly.
- Nothing in the app passes settings into this. The only lever we have is the input picture.

4. Change already in the source: `-c 4096` (measured)

With no `-c`, llama.cpp uses the model's 262,144-token context and reserves a 3 GB KV cache; real usage is ~400 tokens. Same card, byte-identical output every time:

Config	Runs	Avg
baseline (`--no-warmup`)	42.6, 46.0 s	44.3 s
`-c 2048`	36.4, 39.8 s	**38.1 s** (-6.2 s)
`-fit off`	44.6, 45.9 s	45.3 s (no gain, not used)
both	39.8, 40.6 s	40.2 s

Camera-frame simulation (card at 65% of a 1080x1397 frame, 433x560), 4 cards: 48.2 s -> 42.5 s average (-5.7 s), outputs identical. `-c 4096` on card c: 40.3 s (vs 52.6 s baseline, 44.0 s at 2048).

Applied in `MainActivity.kt`: `-c 4096` in `runOnePass()` (CLI) and `ensureServerRunning()` (server), **MiniCPM-V only**. Installed on the phone. It sits on top of the earlier uncommitted handover changes. Untested for `tt_*`/`dl` prompt lengths (should fit in 4096; not verified).

Already disproven in `HANDOVER_MINICPM.md`, not retried: OpenMP pinning, GPU, cropping below 448 px.

5. New approach: field canvas (measured in the harness, not in the app)

Instead of shrinking the whole photo and letting the slicer cut it, cut the printed field regions out of the **original** pixels, stack them into one 448x448 image, and send that. The slicer then sees an image that already fits one tile: 1 encode, **64 image tokens** (prompt 189, output 66, total 255) instead of 3 encodes / 186.

Order of resampling: today = shrink whole photo, then slicer resizes again (twice, shrinking first). Canvas = crop from original, one resize of about 1.0-1.35x.

Regions are **fractions of the front card**, not pixels: PAN number (x 0.20-0.72, y 0.30-0.52), name (0-0.50, 0.52-0.68), father's name (0-0.50, 0.66-0.82), DOB (0-0.50, 0.80-1.00). A first version with tight boxes clipped fields (b: PAN null; d: father `SIR A III`, DOB **invented** as `01/01/1990`).

Real photo, three input variants (SAROJINI M, 848x1058 screenshot)

Variant	Time	Right	Wrong
whole photo, 560 px (what the app does)	38.9 s	4/5	name `SARJINM J`
front card only, 560 px	39.6 s	4/5	PAN `LUQPS8033N`
front card in 2 native-res tiles	33.6 s	4/5	PAN `SARJI1020C` (split cut the number)
field canvas (tuned on this card)	18.1, 20.5 s	5/5	–

Five cards, wider bands, 5 repeats each (25 runs)

Card	Right per run	Wrong (identical every run)
a ROSHAN SAHANI	5/5	–
b SAMAD	5/5	–
c ARJUN MANDLOI	4/5	PAN `FCLPM0017F`, printed `FCLPM0001F`
d ANIZ ISLAM	5/5	–
e SAROJINI M	4/5	name `SAROJIN M`, printed `SAROJINI M`

- Latency 18.9-23.8 s, **average 22.4 s**. First round 19-21 s, later rounds 22-24 s (likely warming up).
- **115/125 fields (92%), but only 15/25 documents fully correct (60%).**
- Decoding is greedy (`--temp 0`): each card gave **one distinct output across all 5 runs**. Repeats show that errors are repeatable, not random. More *cards* are needed, not more runs of these five.
- Ground truth was read by eye from the card images; no verified answer key exists yet.

6. Verdict and risks

Speed gain is real (one tile instead of three). Accuracy is **not yet an improvement at document level** (60% here; the old path was 2/4 in `HANDOVER_MINICPM.md`). Not ready to ship.

1. **Card finding and straightening does not exist.** Fractions only work on an exact front-card crop. Four of five test inputs were already clean crops, one a screenshot. No live camera photo was tested.
2. **Digit slips are the main danger.** `0001` -> `0017` looks valid and is wrong. PAN has no checksum. Format checks (`AAAPL9999A`, 4th char `P` for individuals, 5th char = surname initial) catch letter errors only.
3. **Clipped fields can be invented** (d's DOB). Needs a null-instead-of-guess rule.
4. **One template, standard PAN fronts only.** E-PAN, PVC, older layouts, Aadhaar, DL, passport untested.
5. Card detection will add its own time to the ~22 s; not measured.

Suggested gates before adopting: >= 90% of documents fully correct on 30-50 real phone photos with ground truth checked against the physical cards; no invented values; < 30 s including detection; card detection right on >= 95% of photos. Also: 20 scans in a row for heat and memory.

Next experiments: taller/higher-resolution PAN and name bands on cards c and e (their errors are repeatable, so crop changes may fix them); dedicated PAN-number pass; test with front+back photos.

7. Environment and files (this laptop and phone)

- Toolchain (no sudo, in `~/toolchain/`): JDK 17.0.20.1 (Temurin), Android SDK platform 35 + build-tools 35.0.0 + platform-tools, Gradle 8.7. Build: `JAVA_HOME=~/toolchain/jdk-17* ANDROID_HOME=~/toolchain/android-sdk ~/toolchain/gradle-8.7/bin/gradle --no-daemon :app:assembleDebug` from `home/aiteam/model_testing/android-app/` (needs `local.properties` with `sdk.dir=`). ~2 min.
- `app/src/main/jniLibs/arm64-v8a/` was created and holds the 11 `.so` files (242 MB, copied from `app_build/lib/arm64-v8a/`). The rebuilt APK is 77 MB. **Do not commit `jniLibs/` or `app_build/`** (files over GitHub's 100 MB limit; `app_build/` is ~1.5 GB).
- The uncommitted handover changes compiled with one warning (`MainActivity.kt:1239`).
- **Signing key changed.** The old app was signed with a debug key from another machine (SHA-256 `53d670...`), the new build uses this laptop's (`f0517d...`). Updating in place was impossible, so the app was uninstalled and reinstalled; models and `run_log.txt` were moved aside first and restored. Old and new builds are not update-compatible.
- Backups: `app_build/backup/run_log_phone.txt` (2,532 lines before the reinstall), `run_log_after.txt`. The debug build allows `adb shell run-as com.modeltesting.vlmtester` to read `cache/vlm_input.jpg` (the photo the app actually received).
- Phone harness still at `/data/local/tmp/mini/mt/` (`run.sh`, canvases `w_a..w_e.jpg`, outputs `out_rep_*.log`). Timing gotcha from the previous handover still applies (mksh 32-bit arithmetic).
- Not done: canvas in the app (would go before `downscaleForMiniCpm()`), card detection, server-mode timing in the app, repeat tests on real camera photos.